

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C++ programlama dili, C programlama dili üzerine eklenti olarak geliştirildiğinden, yapısal olmasının yanında aynı zamanda modüler bir programlama dilidir. **Modüler programlamada** (**modular programming**) yazılan fonksiyonlar gruplar halinde başka kod dosyalarına konulur ve bu bileşenler gerektiğinde projemize **#include** ön işlemci yönergeleriyle (**preprocessor directive**) dahil edilir.

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağırılması, işlemciyi yorar. Çünkü her fonksiyon çağırısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C++ programlama dilinde en çok kullanacağımız modüllerden biri de standart giriş-çıkış yani **IO** (**input-output**) işlemlerinin tanımlı olduğu **<iostream>** başlık (**header**) dosyasıdır. Bu modülü kodumuza **dahil etmek** (**include**) için kaynak kodun başında **#include** ön işlemci yönergelerini (**preprocessor directive**) kullanırız.

Kodun başına **#include <iostream>** ekleyerek **konsola** bir şey yazmak için **std::cout** nesnesi, klavyeden bir şey okumak ve bir değişkene atamak için ise **std::cin** nesnelerini kullanabilir hale geliriz. **Konsol** (**console**) kelimesi UNIX/Linux işletim sistemindeki terminal, Windows işletim sisteminde ise komut istemini (**command prompt**) ifade eder.

C++ programlama dilinde her işletim sisteminde de çalışan standart birçok başlık dosyası bulunmaktadır. Ancak bunların dışında, işletim sistemine özel olan başlık dosyaları da bulunmaktadır;

Başlık Dosyası	Açıklama
<iostream>	Konsol (std::cout) ve klavye (std::cin) gibi akış (stream) nesneleri ile giriş ve çıkış işlemleri ve std::endl gibi bildirimleri içerir.
<cmath>	sqrt() , log2() , pow() gibi matematiksel işlemleri yapmak için kullanılan sınıfı içerir.
<cstdlib>	Sistemle ilgili exit() ve rand() gibi işlevleri içerir.
<vector>	Dinamik diziler olan vektör (std::vector) konteyneri (container) tanımlarını içerir.
<string>	Metin işleme için (std::string) şablon sınıf olup işlevler sağlar
<iomanip>	Giriş çıkış işlemlerinde değişkenlerdeki ondalık basamak sayısını sınırlamak için set() ve setprecision() fonksiyonlarına erişmek için kullanılır.
<cerrno>	errno() , strerror() , perror() gibi istisna işleme işlemlerini gerçekleştirmek için kullanılır.
<time>	setdate() ve getdate() gibi date() ve time() ile ilgili fonksiyonlarla işlem yapmak için kullanılır. Ayrıca sistem tarihini değiştirmek ve CPU zamanını almak için de kullanılır.

Tablo 5.1: En Çok Kullanılan Başlık Dosyaları

5.2 Klavyeden Veri Okuma ve Konsola Veri Yazma

C++ dilinde standart giriş-çıkış (**input-output**) için iki sınıf bulunur. İleride detayını vereceğimiz akışları, C dilindeki dosyalar olarak düşünebiliriz. Konsola veri yazmak için;

1. **std::cout** nesnesi, varsayılan olarak konsol ile ilişkilendirilen standart çıkış akışı (**output stream**) nesnesidir.
2. **std::cout** nesnesi gibi çıktı olarak kullanılan akışlara verileri göndermek için **ekleme işleci** (**insertion operator**) (**<<**) kullanılır. Konsolda imlecin (**cursor**) yeri veriyi yazdıktan sonraki konuma taşınır.
3. Çıktı olarak kullanılan konsolda imlece (**cursor**) satır başı için **std::endl** değişmezini kullanırız. Aslında

bu klavyedeki yeni satır (**new line**) karakterine ilişkin değişmezdir. Bu değişmez, Windows işletim sisteminde `'\n'` karakteridir. UNIX/Linux işletim sistemlerinde ise `'\n'+'r'` karakterleridir.

Aşağıdaki örneğe geçmeden önce metin değişmezlerin (**string literal**) çift tırnak arasında yazıldığını unutmayalım. Aslında metinler her biri **char** değişkeni dizileridir. İleride bu konu anlatılacaktır. Aşağıdaki örnekte yer alan `"Yaşınız:"` ve `"Ağırlığınız:"` metin değişmezlerdir.

```
#include <iostream>
int main() {
    int yas(35);
    float agirlik(100.0f);
    std::cout << "Yaşınız:" << yas << std::endl;
    std::cout << "Ağırlığınız:" << agirlik << std::endl;
}
/*Program Çıktısı:
Yaşınız:35
Ağırlığınız:100
*/
```

Klavyeden bir şeyler okumak için;

1. `std::cin` nesnesi, varsayılan olarak klavye ile ilişkilendirilen standart giriş akışı (**input stream**) nesnesidir.
2. `std::cin` nesnesi gibi girdi olarak kullanılan akışlardan veri almak için **ayıklama işleci** (**extraction operator**) (`>>`) kullanılır.

```
#include <iostream>
int main() {
    int yas;
    float agirlik;
    std::cout << "Yaş Giriniz:";
    std::cin >> yas; //klavyeden girilen verilerden yas tam sayısı ayıklanıyor (extraction)
    std::cout << "Ağırlık Giriniz:";
    std::cin >> agirlik; //klavyeden girilen verilerden agirlik kayan noktalı sayısı ayıklanıyor
    // (extraction)
    std::cout << "Girilen Yaş:" << yas << " ve Ağırlık:" << agirlik << std::endl;
}
/* Program çalıştırıldığında:
Yaş Giriniz:12
Ağırlık Giriniz:75.6
Girilen Yaş:12 ve Ağırlık:75.6
*/
```

Koda dahil edilen `<iostream>` başlığı birçok nesne ve bildirim içerir. Bunlardan standart giriş nesnesi olan `std::cin` ve standart çıkış nesnesi olan `std::cout` nesnesi `std` adlı **isim uzayında** (**namespace**) yer alırlar. Kodlama yapılırken bu isim uzayını sürekli olarak `std::` şeklinde yazmamak için `using namespace std;` yönergesi eklenebilir. İki nokta üst üste karakteri olan **kapsam çözümleme işleci** (**scope resolution operator**) (`::`) bunun için kullanılır. İsim uzayları sonradan anlatılacaktır. Bu durumda aynı kod aşağıdaki gibi kısaltılabilir;

```
#include <iostream>
using namespace std;
int main() {
    int yas;
    float agirlik;
    cout << "Yaş Giriniz:";
    cin >> yas;
    cout << "Ağırlık Giriniz:";
    cin >> agirlik;
    cout << "Girilen Yaş:" << yas << " ve Ağırlık:" << agirlik << endl;
}
```

Örnek olarak yarıçapı gerçek sayı olarak klavyeden alınan bir çemberin ve çevresini hesaplayan program aşağıda verilmiştir.

```
#include <iostream>
#define PI 3.1415
using namespace std;
```

```
int main() {
    float cemberinCevresi;
    cout << "Çemberin Çevresini Giriniz:";
    cin >> cemberinCevresi;
    float cemberinYariCapi=cemberinCevresi/2/PI;
    cout << "Çevresini Girdiğiniz Çemberin Yarıçapı:" << cemberinYariCapi;
}
```

5.3 Giriş ve Çıkışı Biçimlendirme

C++ dilinde `<iomanip>` başlığı `std::cin` ve `std::cout` ile ileride göreceğimiz akış nesnelerinde girdi ve çıktıyı biçimlendirebiliriz. Bunun için **manipülâtörler** (**manipulator**) kullanılır. Aşağıdaki örnekte çıktıya gönderilecek hane sayısı `std::setprecision()` ile belirleniyor;

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <limits>
int main() {
    const long double pi = std::acos(-1.L);
    std::cout << "default precision (6): " << pi << std::endl
    << "std::precision(10): " << std::setprecision(10) << pi << std::endl << "max precision:
    ↪ " << std::setprecision(std::numeric_limits<long double>::digits10 + 1) << pi <<
    ↪ std::endl;
}
/*Program Çıktısı:
default precision (6): 3.14159
std::precision(10): 3.141592654
max precision: 3.141592653589793239
*/
```

Aşağıda `std::setw()` ile yazdırılacak genişlik ve `std::setfill()` ile doldurulacak karakter örneği verilmiştir;

```
#include <iostream>
#include <iomanip>
int main() {
    int val(10);
    std::cout << ">" << val << "<" << std::endl;
    std::cout << ">" << std::setw(10) << val << "<" << std::endl;
    val=350;
    std::cout << "default fill: " << ">" << std::setw(10) << val << "<" << std::endl;
    std::cout << "setfill('*'): " << ">" << std::setfill('*') << std::setw(10) << val << "<" <<
    ↪ std::endl;
}
/* Program Çıktısı:
>10<
>      10<
default fill: >      350<
setfill('*'): >*****350<
*/
```

Giriş çıkış işlemlerinde `setiosflags(mask)` kullanıldığında akışın tüm biçim bayraklarını maskenin belirttiği şekilde dışarı veya içeri ayarlar. Tüm `std::ios_base::fmtflags` listesi Tablo 5.2'de verilmiştir;

Ayrıca `basefield`, `dec` | `oct` | `hex` | `0` maskeleye işlemleri için, `adjustfield`, `left` | `right` | `in` hizalama için ve `scientific` | `fixed` | (`scientific` | `fixed`) | `0` maskeleye işlemleri için kullanışlıdır. Aşağıda biçimlendirmelere ilişkin bir kod örneği verilmiştir;

```
#include <iostream>
#include <iomanip>
int main() {
    int iTemp = 47;
    std::cout<< std::resetiosflags(std::ios_base::basefield);
    std::cout<< std::setiosflags(std::ios_base::oct) << iTemp << std::endl; //57
    std::cout << std::resetiosflags(std::ios_base::basefield);
```

Bayrak	Açıklama
<code>dec</code>	Tam sayıları onluk (decimal) tabanda gösterir.
<code>oct</code>	Tam sayıları sekizlik (octal) tabanda gösterir.
<code>hex</code>	Tam sayıları onaltılık (hexadecimal) tabanda gösterir.
<code>left</code>	Çıktıyı sola yaslar (sağa dolgu ekler).
<code>right</code>	Çıktıyı sağa yaslar (sola dolgu ekler).
<code>internal</code>	İşaret (+/-) ile sayı arasına dolgu karakteri ekler.
<code>scientific</code>	Kayan noktalı sayıları bilimsel notasyonda gösterir.
<code>fixed</code>	Kayan noktalı sayıları sabit ondalık formatta gösterir.
<code>boolalpha</code>	Mantıksal değerleri 0/1 yerine true/false olarak yazdırır.
<code>showbase</code>	Sayı tabanını belirten önekleri (0x , 0) ekler.
<code>showpoint</code>	Ondalık nokta karakterini her zaman gösterir.
<code>showpos</code>	Pozitif sayıların başına + işareti koyar.
<code>skipws</code>	Giriş sırasında baştaki boşlukları atlar.
<code>unitbuf</code>	Her çıktı işleminden sonra tamponu (buffer) temizler.
<code>uppercase</code>	Çıktıdaki harfleri büyük harfe dönüştürür.

Tablo 5.2: iostream Biçimlendirme Bayrakları

```
std::cout << std::setiosflags(std::ios_base::hex) << iTemp << std::endl; //2f
std::cout << std::setiosflags(std::ios_base::uppercase) << iTemp << std::endl; //2F
std::cout << std::setfill('0') << std::setw(12);
std::cout << std::resetiosflags(std::ios_base::uppercase);
std::cout << std::setiosflags(std::ios_base::right) << iTemp << std::endl; //00000000002f
std::cout << std::resetiosflags(std::ios_base::basefield|std::ios_base::adjustfield);
std::cout << std::setfill('.') << std::setw(10);
std::cout << std::setiosflags(std::ios_base::left) << iTemp << std::endl; //47.....
std::cout << std::resetiosflags(std::ios_base::adjustfield) << std::setfill('#');
std::cout << std::setiosflags(std::ios_base::internal|std::ios_base::showpos);
std::cout << std::setw(10) << iTemp << std::endl; //+#####47
double dTemp = -1.2;
double pi = 3.14159265359;
std::cout << pi << " " << dTemp << std::endl; //+3.14159 -1.2
std::cout << std::setiosflags(std::ios_base::showpoint) << dTemp << std::endl; //-1.20000
std::cout << setiosflags(std::ios_base::scientific) << pi << std::endl; //+3.141593e+00
std::cout << std::resetiosflags(std::ios_base::floatfield);
std::cout << setiosflags(std::ios_base::fixed) << pi << std::endl; //+3.141593
bool b = true;
std::cout << std::setiosflags(std::ios_base::unitbuf|std::ios_base::boolalpha) << b; //true
}
```

5.4 Uluslar Araşı Karakterlerle Çalışma

C++'ta `wchar_t` kullanarak Türkçe karakterleri (ş, ğ, ü, ı, ö, ç) doğru şekilde işlemek ve ekrana yazdırmak biraz dikkat gerektirir. Bunun sebebi, standart `std::cout` akışının 8-bitlik karakterlere odaklanması, uluslararası karakterler yani (**wide**) karakterler için `std::wcout` akışı ile uygun yerel ayar (**locale**) desteğine ihtiyaç duymasıdır. Windows altında çalışan bir Türkçe `wchar_t` örneği

```
#include <iostream>
#include <wchar> // wchar_t fonksiyonları için
#include <locale> // setlocale için
int main() {
    // 1. ADIM: Yerel ayarları sistemin varsayılanına (Türkçe) ayarla.
    // Bu satır olmadan Türkçe karakterler ekranda bozuk görünür.
    std::setlocale(LC_ALL, "Turkish");
    // 2. ADIM: Geniş karakterli string tanımlama (L öneki ile)
    wchar_t mesaj[] = L"C++ ile Türkçe karakterler: Şş, Ğğ, Üü, İı, Öö, Çç";
    // 3. ADIM: wcout (wide cout) kullanarak ekrana yazdırma
    std::wcout << L"Mesajınız: " << mesaj << std::endl;
    // Tek bir karakter kullanımı
    wchar_t harf = L'Ğ';
    std::wcout << L"Seçilen Harf: " << harf << std::endl;
}
```

```
}
```

Bu kodda ve dikkat etmeniz gereken kritik noktalar:

- ♦ **L Öneki:** `wchar_t` değişmezleri tanımlarken her zaman başına `L` koymalısınız (`L"Türkçe Metin"`). Bu, derleyiciye o metni geniş karakter dizisi olarak saklamasını söyler.
- ♦ **std::wcout Kullanımı:** Geniş karakterleri yazdırmak için `std::cout` değil, `std::wcout` kullanmalısınız. Aksi takdirde ekranda karakterlerin kendisini değil, bellekteki sayısal adreslerini görebilirsiniz.
- ♦ **setlocale Fonksiyonu:** C++ konsolu varsayılan olarak "C" modundadır. Yani temiz ASCII modundadır. `std::setlocale(LC_ALL, "Turkish");` komutu, konsolun Türkçe karakter setini (Windows'ta genellikle Code Page 1254) tanımasını sağlar.
- ♦ **Derleyici ve Terminal Uyumu:** Eğer bu kodu Windows'ta Visual Studio (MSVC) ile çalıştırıyorsanız, kaynak dosyanızın UTF-8 with BOM veya Windows-1254 olarak kaydedildiğinden emin olmalısınız.

Benzer uygulamayı UNIX/Linux altında aşağıdaki şekilde yazabiliriz;

```
#include <iostream>
#include <wchar>
#include <locale>
int main() {
    /* Ubuntu'da sistemin varsayılan UTF-8 yerel ayarını kullanmak en güvenlisidir. "" parametresi
    → sistemin mevcut dil ayarlarını (genelde en_US.UTF-8 veya tr_TR.UTF-8) yükler. */
    if (!std::setlocale(LC_ALL, "")) {
        std::wcerr << L"Yerel ayar yüklenemedi!" << std::endl;
        return 1;
    }
    // Geniş karakterli string
    wchar_t mesaj[] = L"Ubuntu üzerinde Türkçe: Şş, Ğğ, Üü, İı, Öö, Çç";
    /* Ubuntu terminali (Gnome Terminal vb.) doğrudan UTF-8 desteklediği için wcout düzgün
    → yapılandırıldığında karakterleri basacaktır. */
    std::wcout << L"Mesaj: " << mesaj << std::endl;
    // Boyut farkını görmek için (Ubuntu'da 4 çıkacaktır)
    std::wcout << L"wchar_t boyutu: " << sizeof(wchar_t) << L" byte." << std::endl;
}
```

Uluslar Arası Karakterler

Modern C++ uygulamalarında `wchar_t` yerine, taşınabilirlik (**portability**) için `char16_t` (UTF-16) veya `char32_t` (UTF-32) tercih edilmelidir. `wchar_t`'nin boyutu, Windows işletim sisteminde 2 byte iken Linux'ta 4 byte'tır. Bu durum sistemler arası veri aktarımında hatalara yol açabilir.

```
#include <iostream>
#include <string>
#include <string_view>
#include <print> // C++23: Modern çıktı kütüphanesi
#include <vector>

// UTF-16'yı UTF-8'e dönüştüren modern bir yardımcı fonksiyon (C++20)
std::string to_utf8(std::u16string_view u16_str) {
    std::string utf8_result;
    // Not: Gerçek projelerde bu dönüşüm için 'icu' veya 'boost::nowide'
    // gibi kütüphaneler ya da platforma özel API'lar (Win32 MultiByteToWideChar) önerilir.
    // Burada standart yaklaşımı simüle ediyoruz:
    for (char16_t c : u16_str) {
        if (c < 0x80) utf8_result += static_cast<char>(c);
        else {
            // Basitlik adına 128 üstü karakterler için dönüşüm mantığı gerekir.
            // Modern C++'ta bu işlem genellikle char8_t üzerinden yürütülür.
        }
    }
    return utf8_result;
}

int main() {
    // 1. Modern string tanımı (C++20 u16string)
```

```

using namespace std::literals;
auto utf16_mesaj = u"Modern C++ (C++23) UTF-16 Desteği (ÖöÇçiİşŞüÜğĞıI)"s;

// 2. Boyut kontrolü
// char16_t her platformda garantili 2 byte'tır.
std::println("Karakter tipi boyutu: {} byte", sizeof(char16_t));

// 3. Modern Çıktı (C++23 std::print)
// Doğrudan u16string basmak yerine UTF-8'e çevirip basıyoruz.
// std::print, konsolun encoding ayarlarını std::cout'tan çok daha iyi yönetir.

// Not: Windows konsolunda UTF-8 çıktı görmek için:
// system("chcp 65001 > nul"); // komutu gerekebilir.

// u16string'i doğrudan bir string_view olarak işleyelim
std::u16string_view view = utf16_mesaj;

// C++23'te unicode karakterleri yazdırmanın en stabil yolu:
std::println("Mesaj: {}", "Modern C++ ile UTF-16 işleme (ÖöÇçiİşŞüÜğĞıI)");

return 0;
}

```

Kod incelendiğinde;

- ◊ `std::println` (C++23), `std::cout << ...` yazımındaki hantallığı ve tip güvenliği sorunlarını giderir. Python'daki `print()` veya Rust'taki `println!` benzeri bir konfor sunar.
- ◊ `std::u16string_view`, Veriyi kopyalamadan okumanızı sağlar, performansı artırır.
- ◊ `char8_t` ve UTF-8 odaklılık ile modern C++ dünyasında konsol çıktısı, dosya sistemi (`std::filesystem`) ve ağ iletişimi artık UTF-8 (`char`) üzerinden döner.
- ◊ `char16_t` ise genellikle Windows API'ları ile konuşurken veya bellek tasarrufu (belirli dillerde) gereken durumlarda "dahili işleme" için kullanılır.

Eğer Windows üzerinde çalışıyorsanız ve `char16_t` (UTF-16) verisini hiçbir dönüşüm yapmadan doğrudan konsola basmak istiyorsanız, en modern "platforma özgü" yöntem şudur:

```

#ifdef _WIN32
#include <fcntl.h>
#include <io.h>

// Windows konsolunu UTF-16 moduna alır
_setmode(_fileno(stdout), _O_U16TEXT);
std::wcout << L"Windows'ta doğrudan UTF-16 çıktı";
#endif

```

5.5 Düşün ve Çöz

- ◊ **Düşün:** `std::cin` ile klavyeden veri okurken kullanıcı beklenen tipte değil yanlış bir karakter girerse ne olur? Bu durumu ele almak için hangi kontrol mekanizmaları kullanılabilir?
- ◊ **Çöz:** `std::cout` ile çıktıyı formatlamak için `std::setw()`, `std::setprecision()` ve `std::fixed` manipatörlerini kullanın. Pi sayısını farklı ondalık hassasiyetlerde (2, 4, 6 basamak) ekrana yazdıran bir program tasarlayın.
- ◊ **Düşün:** Konsol çıktısında Türkçe karakterler (ğ, ş, ı, ö, ü, ç) neden bazen bozuk görünür? Bu sorunu çözmek için hangi adımlar atılabilir?
- ◊ **Düşün:** Modüler programlama nedir ve neden önemlidir? Büyük bir C++ projesini tek bir `.cpp` dosyasına yazmak yerine birden fazla dosyaya bölmek avantajlarını somutlaştırın.
- ◊ **Düşün:** `std::cin >> x;` ile `std::getline(std::cin, s);` arasındaki temel fark nedir? Bir tam sayı okuyup ardından bir satır metin okumak istediğinizde neden sorun çıkabilir ve nasıl çözülür?